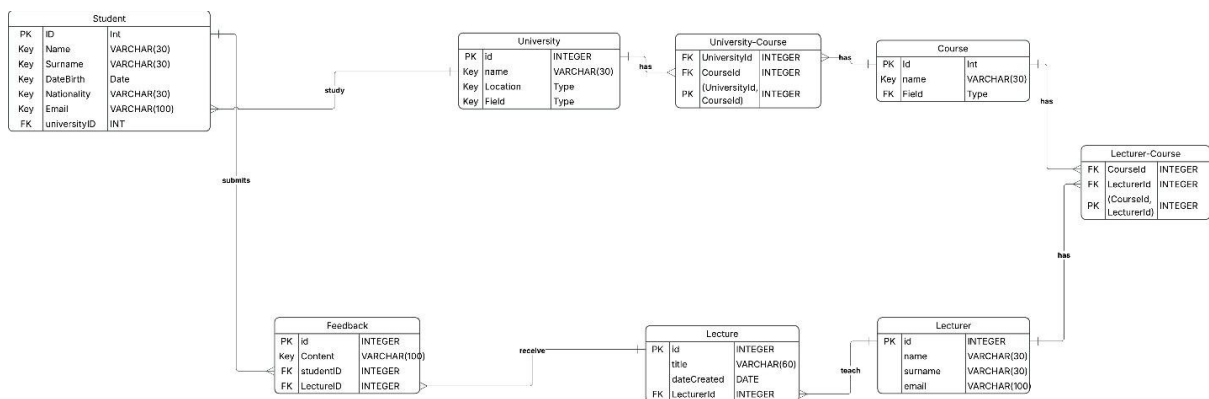


- Define the scope of your societal problem (what should the database be able to answer)?
  - The system to be conceived is aimed at providing feedback on lectures made by lecturers.
- List how the data interacts in short sentences
  - Course has multiple course sessions.
  - A lecturer teaches one or more courses.
  - A Student gives feedback to one or more lectures
  - University provides one or more courses.
- Identify the entities ("nouns", or core objects of your database)
  - Students
  - Lecturers
  - Feedback
  - Courses
  - University
  - Lectures
- Identify the relationships ("verbs", connections between entities)
  - Courses have multiple lectures but lectures have only one course
  - Students give Feedback to Lectures
  - University provides courses
  - Lectures are taught by lecturers
- Use the data interaction descriptions to define the cardinalities of the relationships
  - Course (one) has multiple lectures (many) (one-to-many)
  - Lectures have only one course (one-to-many)
  - Student (one) gives Feedback (one) (one-to-one)
  - Lecture receives feedbacks (one-to-many)
  - University (one) provides courses (many) (one-to-many)
  - Students (many) are enrolled into University (one) (one-to-many)
  - Lecturers work in the University (one-to-many)
- Decide which attributes to include per entity ("columns" per Table)
  - Student: StudentId(PK), StudentName, StudentSurname, StudentDateBirth, StudentNationality, Email
  - Lecturer: LecturerId(PK), LecturerName, LecturerSurname, Email, CoursesId(FK)
  - Feedback: FeedbackId(PK), date, StudentId(FK)
  - Course: CourseId(PK), CourseName

- University: UniversityId(PK), UniversityName, UniversityCity, UniversityState, CourseId(FK)
- Lecture: LectureId(PK), Title, Date, CourseId(FK)

- Define your PKs and FKs
- Each entity requires a unique Primary Key
- Foreign Keys are required to link child tables back to parent tables
- Draw a draft ERD in Crow's foot notation (using any tool you prefer)
- Normalize the ERD:



- 1NF: Atomic Values
- Every cell contains exactly one single, indivisible value (no lists or comma-separated groups), and there must be no repeating groups. For example, the University entity has atomic attributes.

| Id | Name                  |
|----|-----------------------|
| 1  | Maastricht University |
| 2  | Leiden University     |

- 2NF: No Partial Dependencies

Meets 1NF, and every non-key column must depend on the *entire* Primary Key, not just a piece of it. This rule only matters for tables with a composite key. In our ERD these are **University-Course** (UniversityId, CourseId) and **Lecturer-Course** (CourseId, LecturerId).

#### Before (violates 2NF)

Suppose University-Course also stored the university and course names:

| UniversityId (PK) | CourseId (PK) | UniversityName        | CourseName   |
|-------------------|---------------|-----------------------|--------------|
| 1                 | 10            | Maastricht University | Databases    |
| 1                 | 11            | Maastricht University | Graph Theory |
| 2                 | 10            | Leiden University     | Databases    |

UniversityName depends only on UniversityId, and CourseName depends only on CourseId, so each depends on only part of the key. The names are repeated on many rows, and renaming a course means updating several rows.

### After (2NF)

The names are moved to their own tables, and the junction table keeps only the two keys:

*University-Course*

| UniversityId (PK, FK) | CourseId (PK, FK) |
|-----------------------|-------------------|
| 1                     | 10                |
| 1                     | 11                |
| 2                     | 10                |

*University*

| Id (PK) | Name                  |
|---------|-----------------------|
| 1       | Maastricht University |
| 2       | Leiden University     |

*Course*

| Id (PK) | Name         |
|---------|--------------|
| 10      | Databases    |
| 11      | Graph Theory |

Lecturer-Course follows the same rule: it stores only CourseId and LecturerId, while the lecturer's name and email stay in the Lecturer table.

- 3NF: No Transitive Dependencies

Meets 2NF, and non-key columns must not depend on *other* non-key columns. Every attribute should describe the key directly, not something the key points to.

### Before (violates 3NF)

Suppose the Student table also stored details about the student's university:

| Id (PK) | Name | Surname  | UniversityId | UniversityName        | UniversityLocation |
|---------|------|----------|--------------|-----------------------|--------------------|
| 1       | Anna | Rossi    | 1            | Maastricht University | Maastricht         |
| 2       | Tom  | de Vries | 1            | Maastricht University | Maastricht         |
| 3       | Lisa | Bakker   | 2            | Leiden University     | Leiden             |

Here Id → UniversityId → UniversityName, UniversityLocation. The university details depend on UniversityId, which is not the key. They are repeated for every student, and if a university changes its name, every one of its students' rows must be updated or the data becomes inconsistent.

### After (3NF)

Student keeps only the foreign key, and the university details are stored once:

*Student*

| Id (PK) | Name | Surname  | UniversityId (FK) |
|---------|------|----------|-------------------|
| 1       | Anna | Rossi    | 1                 |
| 2       | Tom  | de Vries | 1                 |
| 3       | Lisa | Bakker   | 2                 |

### *University*

| <b>Id (PK)</b> | <b>Name</b>           | <b>Location</b> |
|----------------|-----------------------|-----------------|
| 1              | Maastricht University | Maastricht      |
| 2              | Leiden University     | Leiden          |

Lecture works the same way: it stores LecturerId as a foreign key instead of copying the lecturer's name or email.

The final ERD satisfies 1NF, 2NF and 3NF. The tables above show what the design would look like if these rules were violated, and how separating the data into the University, Course, Student, Lecturer and junction tables removes the redundancy.